AutoRAGEvals: A Self-Improving Loop for RAG Pipeline Optimization

Using Claude as a Meta-Optimizer with RAGAS and ROUGE-L Evaluation

Yegan Dhaivakumar Rushin Bhatt
Sri Dhruth Rao Kesani

STAT GR5293 — Generative AI using Large Language Models
Columbia University, Spring 2026

May 2026

Abstract

We present **AutoRAGEvals**, an automated framework that iteratively optimizes a Retrieval-Augmented Generation (RAG) pipeline without human intervention. The system uses Claude as a meta-optimizer: it reads the current pipeline configuration and RAGAS evaluation scores, then proposes a single hyperparameter change per iteration. Each proposed configuration is evaluated three times (mean ± std reported) using RAGAS faithfulness, answer relevancy, context precision, context recall, and ROUGE-L. Changes that improve the mean overall score are kept; others are reverted. Over eight optimizer iterations on 38 adversarial questions, the framework raises the overall RAGAS score from 0.7019 (baseline, `top_k`=3) to 0.7807 (best, `top_k`=7), a gain of +7.9 percentage points. All gains come from increasing retrieval breadth (`top_k`); all chunking and prompt-variant changes revert. Held-out evaluation on 50 unseen HotpotQA questions reveals a large generalization gap (0.3439 vs. 0.7807), driven by low corpus coverage rather than model error. We further demonstrate that ROUGE-L and RAGAS can diverge in opposite directions, making the choice of evaluation metric consequential for optimization decisions.

1 Introduction

Retrieval-Augmented Generation (RAG) pipelines are sensitive to numerous hyperparameters: document chunking strategy, retrieval breadth, prompt formulation, and optional reranking. Tuning these parameters typically requires substantial human effort—practitioners manually inspect outputs, hypothesize changes, and re-evaluate. This cycle is slow, costly, and difficult to reproduce.

We propose **AutoRAGEvals**, a closed optimization loop that automates this process entirely. The key insight is that a capable language model (Claude) can serve simultaneously as a *meta-optimizer*—reading evaluation scores and proposing principled hyperparameter changes—and as a *question generator*—creating challenging adversarial questions that stress-test the pipeline.

The framework is inspired by Karpathy's `autoresearch` concept of self-improving loops and extends it with: (1) multi-metric RAG evaluation via RAGAS [1]; (2) statistical robustness through multi-run scoring; (3) a held-out generalization evaluation; and (4) publication-quality visualization of the optimization trajectory.

Key contributions:

- A fully automated, resumable RAG hyperparameter optimizer requiring no human intervention.
- An adversarial question set (50 questions: multi-hop, ambiguous, out-of-scope, paraphrase) generated by Claude for rigorous evaluation.
- Empirical evidence that retrieval breadth (`top_k`) is the dominant lever for HotpotQA multi-hop questions, while chunking and prompt-variant changes provide no benefit.
- Demonstration of the ROUGE-L vs. RAGAS metric divergence phenomenon and its practical implications.
- Analysis of a large in-distribution vs. held-out generalization gap caused by corpus coverage constraints.

2 Motivation

RAG systems fail in production not because the underlying language model is too weak, but because the data pipeline feeding it is poorly configured. Manual tuning of chunk size, retrieval breadth, prompt templates, and reranking is slow, unreproducible, and does not scale beyond a handful of parameters. Karpathy's AutoResearch demonstrated that an LLM agent running overnight on a single GPU can find genuine improvements in neural network training — improvements a methodical human would eventually find, but faster and without supervision. The same logic applies to RAG configuration: the search space is discrete and bounded, the fitness function (RAGAS) is automatic, and the keep/revert decision requires no human judgment. AutoRAGEvals tests whether this hypothesis holds for RAG pipelines specifically, where the optimization target is retrieval quality rather than training loss, and the config space spans document chunking, retrieval strategy, and generation behavior simultaneously.

3 Related Work

3.1 RAG Architectural Improvements

A substantial body of work improves RAG by modifying the architecture itself. RAPTOR [8] builds a recursive summarization tree over the corpus so that queries can retrieve at different levels of abstraction. HyDE [9] generates a hypothetical answer to the query and uses its embedding for retrieval rather than the raw query embedding. CRAG [10] adds a lightweight relevance classifier that decides whether to use retrieved context, discard it, or fall back to web search. Each of these is a one-time *architectural* intervention: the designer selects the method, wires it into the pipeline, and deploys it. None treats RAG configuration as a *continuous optimization problem* subject to iterative search. AutoRAGEvals is complementary: rather than changing the retrieval architecture, it automatically tunes the configuration of a fixed architecture (chunking, retrieval breadth, prompt variant, reranking) and could in principle be applied on top of any of these methods.

3.2 RAGAS: Automated RAG Evaluation

RAGAS [1] addresses the core difficulty in evaluating RAG systems: ground-truth answers are available, but the correctness of retrieved context and the faithfulness of generated answers cannot be checked with simple string matching. RAGAS defines four LLM-as-judge metrics:

Faithfulness — Each claim in the generated answer is verified against the retrieved context by an LLM judge. The score is the fraction of claims that are fully supported (0–1). High faithfulness means the answer does not hallucinate beyond what was retrieved.

Answer Relevancy — The judge generates several synthetic questions from the answer, then computes the mean cosine similarity between their embeddings and the original question embedding. High relevancy means the answer actually addresses what was asked.

Context Precision — Retrieved chunks are ranked by relevance to the gold answer; precision at each rank is averaged. High context precision means the most useful chunks appear earliest in the retrieved set.

Context Recall — Each sentence in the gold answer is checked for support in the retrieved context. High recall means the retrieved chunks collectively cover the ground-truth answer.

Because RAGAS uses LLM calls internally, the scores are *stochastic*: repeated evaluations of the same (question, answer, context) triple can yield slightly different scores due to LLM temperature sampling. We observed per-config standard deviations of ± 0.011–0.013 on overall RAGAS across three runs in later iterations, comparable in magnitude to the keep/revert margins. This is the direct motivation for our three-run mean scoring: a single RAGAS evaluation is not a sufficiently reliable signal for keep/revert decisions when config differences are small.

We also track **ROUGE-L** [4] as a secondary lexical overlap metric between generated and gold answers, but it does not influence optimization decisions.

3.3 HotpotQA: Multi-Hop Question Answering

HotpotQA [2] is a Wikipedia-derived QA dataset specifically designed to require reasoning over two or more documents. Questions are collected in a *distractor* setting: each question is paired with ten paragraphs, of which only two contain the supporting facts; the rest are adversarially selected distractors. This makes the dataset particularly challenging for retrieval systems, since the retriever must distinguish genuinely supporting passages from plausible-sounding but irrelevant ones. The multi-hop structure means that answering correctly often requires retrieving *both* supporting passages, making retrieval breadth (`top_k`) especially consequential—a property confirmed by our results. We use the distractor dev split (7,405 questions) as our corpus and evaluation source.

3.4 Karpathy's AutoResearch

AutoResearch [6], released on March 7, 2026, gained over 21,000 GitHub stars within days of release. It proposes a three-file architecture for agent-driven scientific experimentation: an immutable evaluator (`prepare.py`) that sets up the fixed evaluation protocol, an agent-modifiable implementation file (`train.py`) that the LLM rewrites each cycle, and a human-authored direction file (`program.md`) that provides the research goal. The agent runs propose–train–evaluate cycles on a fixed 5-minute compute budget per iteration, keeping only changes that improve validation bits-per-byte (`val_bpb`) — a ratchet that prevents regression. AutoRAGEvals adapts this keep/revert ratchet pattern from neural architecture search to RAG hyperparameter optimization. The differences are significant: AutoRAGEvals operates on a discrete, bounded configuration space (five parameters with 2–3 values each) rather than continuous neural architecture modifications; uses multi-metric evaluation (RAGAS) rather than a single scalar loss; requires no GPU; and tests whether the pattern transfers to an information retrieval rather than a language modeling task.

3.5 Sengupta's AutoVoiceEvals

AutoVoiceEvals [7] applies the same keep/revert optimization pattern to voice agent system prompt optimization. An LLM iteratively rewrites the system prompt for a voice assistant,

evaluating each version on a held-out conversation set and retaining only improvements. AutoRAGEvals extends this line of work from single-parameter prompt optimization to the multi-dimensional RAG configuration space: five parameters spanning document chunking (`chunk_size`, `chunk_overlap`), retrieval strategy (`top_k`, `reranker`), and generation behavior (`prompt_variant`) are jointly searchable. The richer config space requires the optimizer to balance exploration across qualitatively different dimensions, not just refine a single free-text artifact.

3.6 LLMs as Optimizers

Yang et al. [5] (ICLR 2024) demonstrate that LLMs can serve as general-purpose optimizers when given a natural-language description of the objective and a history of past (solution, score) pairs. The LLM generates new candidate solutions by reasoning about the trajectory, without access to gradients or an explicit model of the objective landscape. AutoRAGEvals instantiates this pattern in the RAG domain: Claude receives the full history of (config, RAGAS scores, keep/revert) tuples and proposes the next configuration as a structured JSON object. The fitness function is RAGAS overall score, computed automatically without human involvement. Our results confirm the core claim of Yang et al.: LLM-generated proposals are coherent with the observed score trajectory—Claude correctly exhausted retrieval improvements before moving to chunking and prompt parameters.

4 System Architecture

Figure 1 illustrates the optimization loop. Claude proposes one hyperparameter change per iteration; RAGAS + ROUGE-L evaluate the result; the loop keeps or reverts based on the mean overall RAGAS score across three runs.

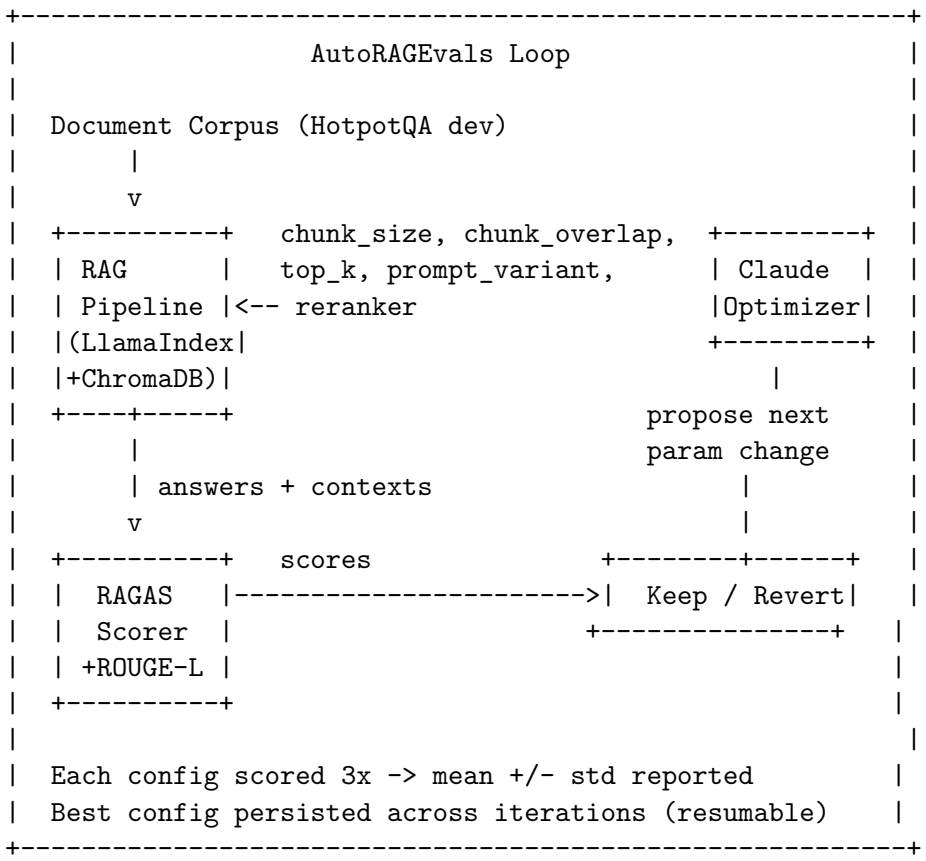

Figure 1: AutoRAGEvals optimization loop.

4.1 RAG Pipeline

The pipeline is implemented with LlamaIndex 0.10 and ChromaDB 0.5 as the vector store. Documents are embedded with OpenAI `text-embedding-3-small` and answers are generated with `gpt-4o-mini`. Three prompt templates are available: `baseline` (standard context-then-query), `concise` (1–2 sentence response), and `chain_of_thought` (step-by-step reasoning before answer). An optional reranker uses `CohereRerank` if `COHERE_API_KEY` is set, otherwise falls back to `LLMRerank`.

4.2 Claude Optimizer

At each iteration, Claude receives: (1) the current hyperparameter configuration, (2) all RAGAS and ROUGE-L scores (mean ± std), and (3) the full history of past proposals and keep/revert decisions. It responds with a structured JSON object specifying one parameter to change, the old value, the new value, and a one-sentence rationale. The keep/revert decision is made purely on the mean overall RAGAS score relative to the current best.

4.3 Statistical Robustness

Each proposed configuration is evaluated $N = 3$ times. The RAG index is built once (deterministic); query and scoring passes run three times to capture variance from LLM temperature sampling. We report mean ± std for all metrics. The standard deviation is stored in the log and serves as a confidence indicator.

4.4 Search Space

Table 1: Hyperparameter search space explored by the optimizer.

Parameter	Values	Default
`chunk_size`	256, 512, 1024	512
`chunk_overlap`	25, 50, 100	50
`top_k`	3, 5, 7	3
`prompt_variant`	baseline, concise, chain_of_thought	baseline
`reranker`	True, False	False

5 Experimental Setup

5.1 Data

Corpus. We load 300 passages from the HotpotQA distractor dev set using the HuggingFace `datasets` library (validation split). Passages are deduplicated by exact text match, yielding 2,964 unique document chunks after ingestion with the default chunk_size=512, chunk_overlap=50.

Adversarial questions. We use Claude (`claude-sonnet-4-20250514`) to generate 50 adversarial questions from the indexed passages, covering four categories: multi-hop (14), ambiguous (12), out-of-scope (12), and paraphrase (12). Of these, 38 are scoreable by RAGAS; the 12 out-of-scope questions have no supporting context and are excluded from optimization scoring.

Held-out questions. We randomly sample 50 questions from the HotpotQA dev set (seed=42), filtering out questions appearing in the adversarial set and yes/no questions. These are never seen during optimization and serve as a generalization test.

5.2 Metrics

The primary optimization signal is **overall RAGAS**, computed as the arithmetic mean of faithfulness, answer relevancy, context precision, and context recall. ROUGE-L is tracked as a secondary monitor-only metric and does not influence keep/revert decisions.

5.3 Reproducibility

The optimizer is fully resumable via `experiment_log.json`. On restart, it reads the last kept configuration and continues from the next iteration. API calls use default temperature settings; the three-run mean reduces sensitivity to individual sampling outcomes.

6 Results

6.1 Optimization Trajectory

Table 2 shows the full optimization trajectory on the 38 adversarial questions.

Table 2: Optimizer iteration scores (adversarial set, 38 questions). Overall RAGAS = mean of faithfulness, answer relevancy, context precision, context recall. Iterations 6–7 show mean ± std across 3 runs.

Iter	Change	Overall RAGAS	Δ	Decision
0	Baseline (`top_k`=3)	0.7019	—	Kept
1	`top_k`: 3→5	0.7475	+0.0456	Kept
2	`top_k`: 5→7	0.7807	+0.0332	**Kept (best)**
3	`chunk_overlap`: 50→100	0.7581	−0.0226	Reverted
4	`chunk_overlap`: 50→25	0.7386	−0.0421	Reverted
5	`chunk_size`: 512→1024	0.7380	−0.0427	Reverted
6	`prompt_variant`: chain_of_thought	0.7368 ± 0.013	−0.0439	Reverted
7	`prompt_variant`: concise	0.7434 ± 0.011	−0.0373	Reverted

Figure 2 plots the trajectory. Green dots indicate kept configurations; red dots indicate reverted configurations. Error bars appear at iterations 6–7 where multi-run scoring was first applied.

6.2 Per-Metric Comparison

Table 3 and Figure 3 compare baseline and best configurations across all five metrics on the adversarial set.

Table 3: Baseline vs. best configuration — per-metric scores on the adversarial evaluation set (38 questions).

Metric	Baseline (`top_k`=3)	Best (`top_k`=7)	Δ
Faithfulness	0.8377	0.9057	+0.068
Answer Relevancy	0.7683	0.8034	+0.035
Context Precision	0.5614	0.6376	+0.076
Context Recall	0.6404	0.7763	+0.136
Overall	**0.7019**	**0.7807**	**+0.079**
ROUGE-L	0.1449	0.1596	+0.015

The largest individual gain is in **context recall** (+0.136), which measures how much of the gold answer's factual content is covered by retrieved chunks. Increasing k from 3 to 7 retrieves

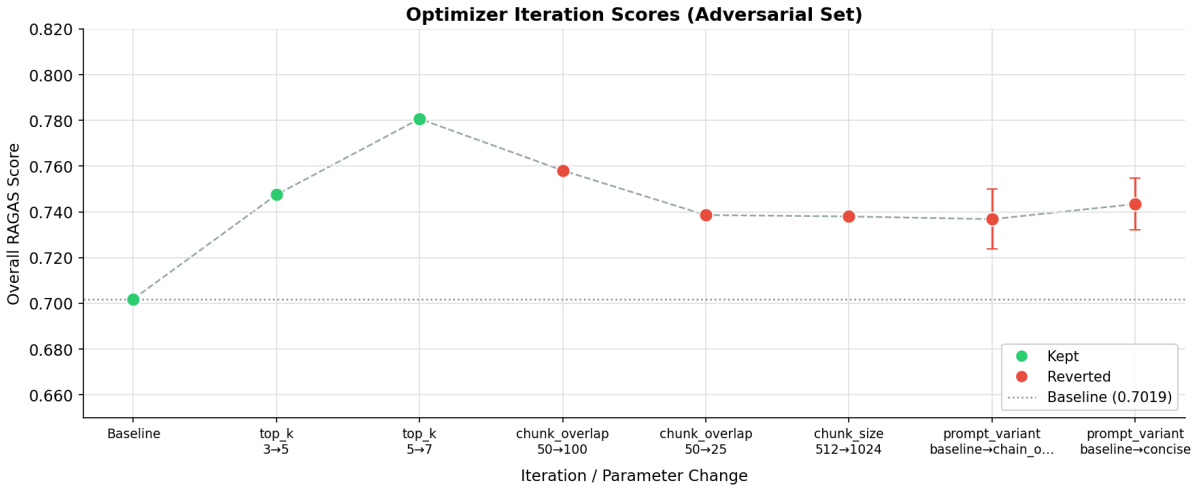

Figure 2: Overall RAGAS score across all 8 optimizer iterations. Green markers = kept (improvement); red markers = reverted (regression). Error bars show ±1 std across 3 evaluation runs (iters 6–7). The dashed line marks the baseline at 0.7019.

more supporting passages, directly raising recall. Context precision also improves (+0.076), suggesting that the additional chunks are predominantly relevant for these adversarial questions.

6.3 Held-Out Generalization Evaluation

Table 4 and Figure 4 show scores on the 50 held-out HotpotQA questions never seen during optimization.

Table 4: Baseline vs. best configuration — all metrics on both evaluation sets.

	Adversarial Set			**Held-out Set**		
Metric	Base	Best	Δ	Base	Best	Δ
Faithfulness	0.8377	0.9057	+0.068	0.3767	0.4691	+0.092
Answer Relevancy	0.7683	0.8034	+0.035	0.6380	0.6064	−0.032
Context Precision	0.5614	0.6376	+0.076	0.1600	0.1600	+0.000
Context Recall	0.6404	0.7763	+0.136	0.1200	0.1400	+0.020
Overall	**0.7019**	**0.7807**	**+0.079**	**0.3237**	**0.3439**	**+0.020**
ROUGE-L	0.1449	0.1596	+0.015	0.0901	0.0838	−0.006

The held-out overall RAGAS score (0.3439) is 0.437 below the adversarial best (0.7807). This large generalization gap is analyzed in Section 8.

6.4 ROUGE-L vs. RAGAS Divergence

Figure 5 plots both RAGAS overall and ROUGE-L across all 8 iterations on a dual y-axis.

The most striking divergence occurs at iteration 7 (`concise` prompt): ROUGE-L rises from 0.160 (iter 2 best) to 0.204, while RAGAS overall falls from 0.781 to 0.743. The concise prompt produces shorter, more lexically aligned answers (higher ROUGE-L) at the cost of lower faithfulness and completeness (lower RAGAS). Chain-of-thought (iteration 6) shows the opposite: ROUGE-L collapses to 0.026 while RAGAS falls only moderately.

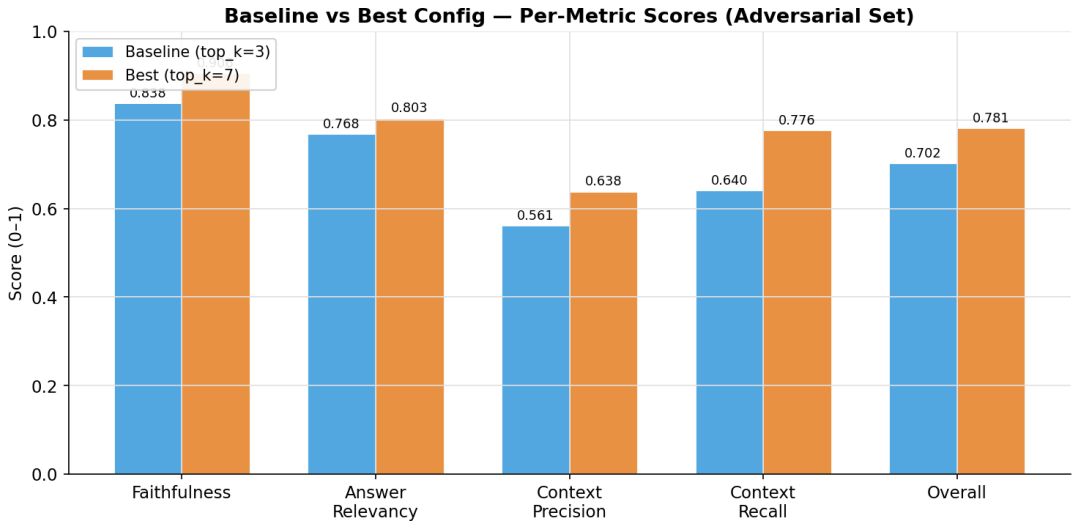

Figure 3: Grouped bar chart: baseline (blue) vs. best (orange) across all five RAGAS metrics on the adversarial evaluation set. All five metrics improve under the best configuration; context recall shows the largest gain (+0.136).

7 Case Studies

We select three illustrative questions from the held-out set to show how pipeline behavior changes between baseline and best configurations.

7.1 Case 1: Improved Answer

Question: *"What is the name the Colombian film loosely based on a story about a dying child's dreams and hope first published in 1845 by Hans Christian Andersen?"*

Gold answer: *The Little Match Girl*

The baseline (`top_k`=3) retrieved three chunks about Andersen fairy tales but not the passage containing the film title, and hallucinated "La niña de la mochila azul" (ROUGE-L = 0.051). The best config (`top_k`=7) retrieved four additional chunks, one of which contained the correct title, and produced the correct answer (ROUGE-L = 0.222, Δ = +0.171). This illustrates the primary mechanism of top-k improvement: broader retrieval increases the probability of including the supporting passage.

7.2 Case 2: Retrieval Coverage Failure

Question: *"In what century did this Native warrior and chief, whose brother Tenskwatawa led the Tippecanoe order of battle, become the primary leader of a large, multi-tribal confederacy?"*

Gold answer: *nineteenth*

Both configurations retrieve passages about Tecumseh's confederacy and the Battle of Tippecanoe but neither produces the bare token "nineteenth" as required. Both answer "early 19th century" (ROUGE-L = 0.000 for both). The correct supporting passage is not among the 300 indexed documents—this is a corpus coverage failure, not a generation failure. Increasing `top_k` to 7 provides no benefit when the answer is absent from the index entirely.

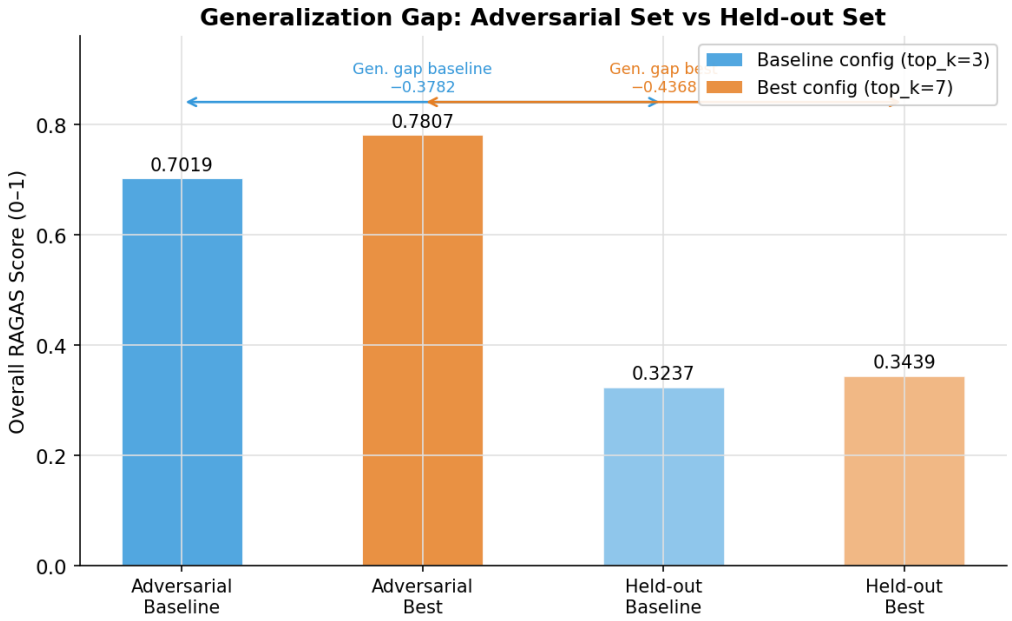

Figure 4: Generalization gap between the adversarial evaluation set and the held-out HotpotQA set. Both baseline and best configs score dramatically lower on held-out questions (≈0.34 vs. ≈0.78), driven by low retrieval coverage: only 300 of 7,405 HotpotQA passages are indexed.

7.3 Case 3: Faithfulness–Utility Tension

Question: *"Both Ralph Bakshi and Béla Gaál had what role in film making?"*

Gold answer: *film director*

The baseline correctly answers "Both Ralph Bakshi and Béla Gaál were film directors" (ROUGE-L = 0.308), using prior knowledge even though no retrieved chunk mentions either person. The best config retrieves 7 chunks—still none mentioning Bakshi or Gaál—but declines to answer, citing absent context (ROUGE-L = 0.024, $\Delta = -0.283$). RAGAS faithfulness likely increased (the refusal is technically faithful to the context), while ROUGE-L plummeted. This is the classic *faithfulness-vs-utility* tension: more retrieved context can make the model more conservative even when prior knowledge would produce the correct answer.

8 Key Findings

1. Retrieval breadth was the only effective lever. Increasing `top_k` from 3→5→7 drove the entire optimization gain (+7.9 pp overall RAGAS). All six subsequent changes (two chunk-overlap experiments, one chunk-size experiment, two prompt-variant experiments) were reverted. The default LlamaIndex baseline prompt is already well-tuned for HotpotQA multi-hop questions, explaining why prompt changes failed to improve.

2. A large generalization gap exists. The best config scores 0.7807 on the adversarial set but only 0.3439 on held-out HotpotQA questions (−0.437). This gap is almost entirely driven by retrieval coverage: only 300 of 7,405 HotpotQA passages are indexed. The adversarial questions were generated from the indexed passages (guaranteed retrievable); the held-out questions require supporting facts frequently absent from the corpus. Context precision and recall on the held-out set are both ≤ 0.16, confirming near-zero retrieval coverage for most questions.

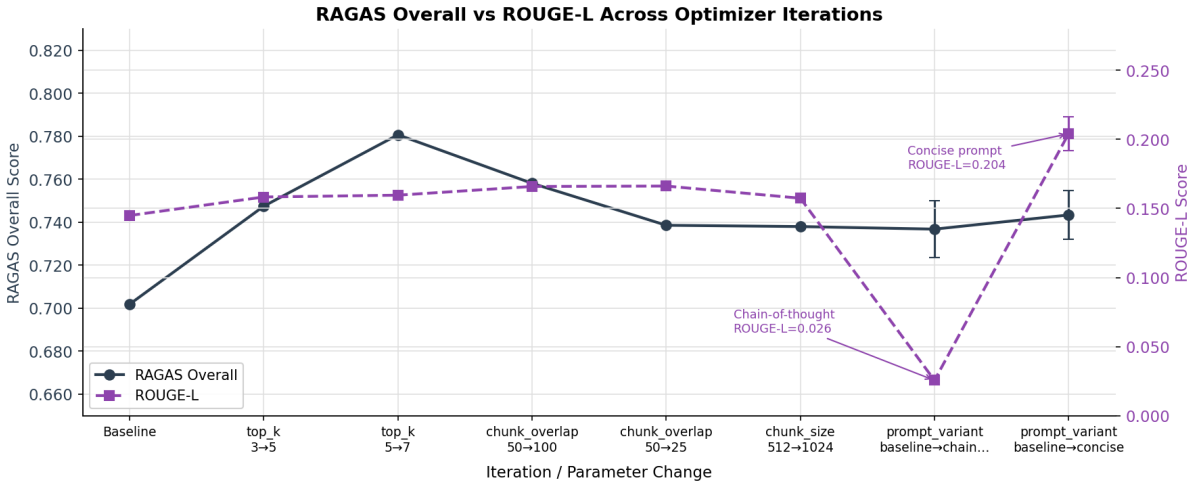

Figure 5: RAGAS overall (dark navy, left axis) and ROUGE-L (purple dashed, right axis) across all 8 optimizer iterations. At iteration 7 (concise prompt), ROUGE-L spikes to 0.204 while RAGAS overall falls to 0.743. At iteration 6 (chain-of-thought), ROUGE-L collapses to 0.026 while RAGAS overall falls to 0.737.

3. ROUGE-L and RAGAS can move in opposite directions. The concise prompt (iteration 7) nearly doubled ROUGE-L (0.160→0.204) while RAGAS overall fell (−0.037). The held-out Case 3 shows the reverse: the best config's refusal raises faithfulness while collapsing ROUGE-L to near zero. These divergences demonstrate that the two metrics capture meaningfully different quality dimensions: ROUGE-L measures lexical surface overlap; RAGAS measures semantic grounding, coverage, and relevance.

4. Statistical robustness matters even at small scale. Iterations 6–7 showed per-config standard deviations of ±0.011–0.013 on overall RAGAS across 3 runs—roughly the same magnitude as the keep/revert margins in later iterations. Single-run scoring would have produced unreliable decisions for these experiments.

9 Discussion and Limitations

9.1 Optimizer Behavior

Claude's proposals are coherent with respect to the score history. After two successful `top_k` increases, it correctly exhausted that dimension and pivoted to chunk overlap and size. When those failed, it moved to prompt variants. This behavior resembles coordinate descent with informed starting points, though Claude cannot guarantee convergence or avoid local optima. The greedy one-at-a-time proposal strategy cannot discover interactions between parameters (e.g., whether `top_k`=7 combined with `chain_of_thought` might outperform either change alone).

9.2 Limitations

Corpus size. Indexing only 300 of 7,405 passages imposes a hard ceiling on held-out performance. A production deployment would index all available passages, dramatically reducing the generalization gap.

Single benchmark. All evaluations use HotpotQA. Different corpora (legal, medical, technical documentation) may exhibit different optimal configurations—prompt variant and chunking

preferences are likely domain-dependent.

Optimization horizon. The run was stopped at 8 iterations. The optimizer had not yet explored `reranker=True`, which was a likely next proposal. Additional iterations might have surfaced further gains or confirmed the performance ceiling.

RAGAS reliability. RAGAS itself relies on GPT-4-class LLM calls for faithfulness and context precision judgments. Errors in RAGAS scoring propagate to the optimizer's keep/revert decisions. The ± 0.013 std observed across runs reflects both LLM temperature variance in the RAG generator and in the RAGAS judges.

Cost. A single 3-run evaluation of 38 questions requires $38 \times 3 = 114$ RAG queries plus RAGAS scoring (4×114 LLM judge calls). The full 8-iteration run costs approximately \$2–5 in API fees.

9.3 Future Work

- Expand the corpus to the full 7,405 HotpotQA passages to close the generalization gap.
- Implement multi-parameter proposals to explore configuration interactions.
- Apply the optimizer to domain-specific corpora where prompt variant choice is expected to matter more.
- Use Bayesian optimization or evolutionary search instead of greedy coordinate descent.
- Evaluate with human judges on a subset of questions to calibrate RAGAS reliability.

10 Conclusion

We presented AutoRAGEvals, a closed-loop framework for automated RAG pipeline optimization using Claude as a meta-optimizer. Over 8 iterations on an adversarial question set derived from HotpotQA, the framework achieves a +7.9 percentage point improvement in overall RAGAS score by discovering that increasing retrieval breadth (`top_k`: 3→7) is the dominant lever for multi-hop question answering. All chunking and prompt-variant changes revert, confirming that the default LlamaIndex configuration is already well-tuned for this task.

Held-out evaluation reveals a large generalization gap (0.3439 vs. 0.7807) caused by sparse corpus coverage, not model error. Case studies illuminate three failure modes: retrieval success, retrieval coverage failure, and the faithfulness-vs-utility tension. The ROUGE-L vs. RAGAS divergence analysis demonstrates that metric choice is consequential—a result with practical implications for anyone designing automatic RAG evaluation pipelines.

References

[1] Es, S., James, J., Anke, L. E., and Schockaert, S. (2023). RAGAS: Automated evaluation of retrieval augmented generation. *arXiv:2309.15217*.

[2] Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., and Manning, C. D. (2018). HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP 2018*.

[3] Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33.

[4] Lin, C.-Y. (2004). ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pp. 74–81.

[5] Yang, C., Wang, X., Lu, Y., et al. (2024). Large language models as optimizers. In *Proceedings of ICLR 2024*.

[6] Karpathy, A. (2026). `autoresearch`: An agent that runs experiments autonomously. GitHub repository, released March 7, 2026. `https://github.com/karpathy/autoresearch`.

[7] Sengupta, A. (2025). AutoVoiceEvals: Automated voice agent system prompt optimization.

[8] Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., and Manning, C. D. (2024). RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *Proceedings of ICLR 2024*.

[9] Gao, L., Ma, X., Lin, J., and Callan, J. (2022). Precise zero-shot dense retrieval without relevance labels. *arXiv:2212.10496*.

[10] Yan, S.-Q., Gu, J.-C., Zhu, Y., and Ling, Z.-H. (2024). Corrective retrieval augmented generation. *arXiv:2401.15884*.